

Esercizio 3 Navigare nel Filesystem Linux e Impostazioni dei Permessi Obiettivi:

In questo laboratorio, prenderai familiarità con i filesystem Linux.

- Parte 1 Esplorare i Filesystem in Linux
- Parte 2 Permessi dei File
- Parte 3 Link Simbolici e Altri Tipi di File Speciali Risorse Richieste
- VM CyberOps Workstation

I filesystem devono essere montati prima di poter essere accessibili e utilizzati. In informatica, montare un filesystem significa renderlo accessibile al sistema operativo. Montare un filesystem è il processo di collegare la partizione fisica sul dispositivo a blocchi (hard disk, unità SSD, pen drive, ecc.) a una directory, attraverso la quale è possibile accedere all'intero filesystem. Poiché la suddetta directory diventa la radice del filesystem appena montato, è anche conosciuta come punto di montaggio Usiamo il comando lsblk per visualizzare tutti i dispositivi a blocchi.

```
[analyst@secOps ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0   10G  0 disk
└─sda1       8:1    0   10G  0 part /
sdb          8:16   0    1G  0 disk
└─sdb1       8:17   0 1023M  0 part
sr0         11:0    1 1024M  0 rom
```

Usiamo il comando mount per visualizzare informazioni più dettagliate sui filesystem attualmente montati nella VM CyberOps Workstation.

```
[analyst@secOps ~]$ mount
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
sys on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
dev on /dev type devtmpfs (rw,nosuid,relatime,size=995444k,nr_inodes=248861,mode=755,inode64)
run on /run type tmpfs (rw,nosuid,nodev,relatime,mode=755,inode64)
/dev/sda1 on / type ext4 (rw,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=41,pgrp=1,timeout=0,minproto=5,maxproto=5,direct,pipe_ino=1828)
hugetlbfs on /dev/hugepages type hugetlbfs (rw,nosuid,nodev,relatime,pagesize=2M)
tracefs on /sys/kernel/tracing type tracefs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /tmp type tmpfs (rw,nosuid,nodev,nr_inodes=1048576,inode64)
mqueue on /dev/mqueue type mqueue (rw,nosuid,nodev,noexec,relatime)
debugfs on /sys/kernel/debug type debugfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /run/credentials/systemd-journald.service type tmpfs (ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr_inodes=1024,mode=700,inode64,noswap)
configfs on /sys/kernel/config type configfs (rw,nosuid,nodev,noexec,relatime)
fusectl on /sys/fs/fuse/connections type fusectl (rw,nosuid,nodev,noexec,relatime)
tmpfs on /run/credentials/systemd-networkd.service type tmpfs (ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr_inodes=1024,mode=700,inode64,noswap)
tmpfs on /run/user/1000 type tmpfs (rw,nosuid,nodev,relatime,size=200916k,nr_inodes=50229,mode=700,uid=1000,gid=1000,inode64)
```

Eseguiamo di nuovo il comando mount, ma questa volta, usa la pipe | per inviare l'output di mount a grep per filtrare l'output e visualizzare solo il filesystem radice:

```
[analyst@secOps ~]$ mount | grep sda1
/dev/sda1 on / type ext4 (rw,relatime)
[analyst@secOps ~]$
```

Nell'output filtrato sopra, mount ci mostra che il filesystem radice si trova nella prima partizione del dispositivo a blocchi sda (/dev/sda1. Sappiamo che questo è il filesystem radice a causa del punto di montaggio usato: "/" (il simbolo della barra). L'output ci dice anche il tipo di formattazione usato nella partizione, ext4 in questo caso. Le informazioni tra parentesi si riferiscono alle opzioni di montaggio della partizione.

Eseguiamo cd / e ls -l.

```
[analyst@secOps ~]$ cd /
[analyst@secOps /]$ ls -l
total 52
lrwxrwxrwx  1 root root    7 May  3  2025 bin -> usr/bin
drwxr-xr-x  3 root root 4096 Jun 18 19:07 boot
drwxr-xr-x 20 root root 3920 Dec  9 04:16 dev
drwxr-xr-x 73 root root 4096 Jun 19 04:45 etc
drwxr-xr-x  3 root root 4096 Mar 20  2018 home
lrwxrwxrwx  1 root root    7 May  3  2025 lib -> usr/lib
lrwxrwxrwx  1 root root    7 May  3  2025 lib64 -> usr/lib
drwx----- 2 root root 16384 Mar 20  2018 lost+found
drwxr-xr-x  2 root root 4096 Jan  5  2018 mnt
drwxr-xr-x  3 root root 4096 Jun 17 15:07 opt
dr-xr-xr-x 203 root root    0 Dec  9 04:16 proc
drwxr-xr-x  8 root root 4096 Jun 18 20:09 root
drwxr-xr-x 22 root root  600 Dec  9 04:32 run
lrwxrwxrwx  1 root root    7 May  3  2025/sbin -> usr/bin
drwxr-xr-x  6 root root 4096 Mar 24  2018 srv
dr-xr-xr-x 13 root root    0 Dec  9 04:16 sys
drwxrwxrwt 12 root root  280 Dec  9 05:17 tmp
drwxr-xr-x 10 root root 4096 Jun 19 03:15 usr
drwxr-xr-x 12 root root 4096 Jun 19 04:45 var
[analyst@secOps /]$
```

L'output mostra il contenuto della directory radice "/". Queste sono le directory principali del filesystem, come bin, boot, dev, etc, home, usr, var e così via. Questo elenco non rappresenta dispositivi o partizioni ma solo le cartelle che compongono il filesystem già montato. I file e le directory elencati sono fisicamente memorizzati sulla partizione che contiene il filesystem root, cioè quella montata su "/". Quindi tutto ciò che vediamo proviene dalla partizione principale del

sistema, di solito qualcosa come /dev/sda1 o una partizione equivalente, non da tutti i dispositivi presenti.

La partizione /dev/sdb1 non viene mostrata perché il comando eseguito è semplicemente “ls -l /” che mostra il contenuto della directory root, non i dispositivi disponibili né le partizioni montate. Una partizione compare nel filesystem solo se viene montata in una directory specifica. Se /dev/sdb1 non è montata, non apparirà da nessuna parte quando fai ls. Per vederla dovresti controllare con comandi come lsblk, fdisk -l o mount. Se non è montata, non fa parte del filesystem visibile e quindi non compare nell’output mostrato.

Usiamo il comando ls -l per verificare che la directory second_drive sia nella directory home dell'utente analyst.

```
analyst@secOps ~]$ ls -l second_drive/
total 0
analyst@secOps ~]$
```

Notiamo che la directory è vuota.

Usiamo il comando mount per montare /dev/sdb1 sulla directory second_drive appena creata. La sintassi di mount è: mount [opzioni] <dispositivo_da_montare> <punto_di_montaggio>.

```
total 0
[analyst@secOps ~]$ sudo mount /dev/sdb1 ~/second_drive/
[sudo] password for analyst:
[analyst@secOps ~]$
```

. Ora che /dev/sdb1 è stato montato su /home/analyst/second_drive, usiamo ls -l per elencare di nuovo il contenuto della directory.

```
[analyst@secOps ~]$ ls -l second_drive/
total 20
drwx----- 2 root    root    16384 Mar 26  2018 lost+found
-rw-r--r--  1 analyst analyst  183 Mar 26  2018 myFile.txt
[analyst@secOps ~]$
```

La directory non è più vuota perché abbiamo montato la partizione /dev/sdb1 dentro quella directory. Prima la cartella second_drive era semplicemente una directory normale nel filesystem principale e quindi risultava vuota. Dopo il comando di mount, il contenuto della partizione /dev/sdb1 viene mostrato all'interno di quella directory, quindi ora compaiono lost+found e myFile.txt. I file elencati non si trovano nel disco principale ma sono fisicamente memorizzati sulla partizione /dev/sdb1. Il mount rende semplicemente accessibile quel contenuto all'interno della directory second_drive, ma i dati risiedono sul secondo disco o sulla partizione montata.

Eseguiamo di nuovo il comando mount senza opzioni per visualizzare informazioni dettagliate sulla partizione /dev/sdb1. Come prima, usa il comando grep per visualizzare solo i filesystem /dev/sdX:

```
[analyst@secOps ~]$ mount | grep /dev/sd
/dev/sda1 on / type ext4 (rw,relatime)
/dev/sdb1 on /home/analyst/second_drive type ext4 (rw,relatime)
[analyst@secOps ~]$
```

Smontare i filesystem è altrettanto semplice. Assicuriamoci di cambiare la directory in qualcosa al di fuori del punto di montaggio e usa il comando umount, come mostrato sotto:

```
[analyst@secOps ~]$ sudo umount /dev/sdb1
[analyst@secOps ~]$ ls -l second_drive/
total 0
[analyst@secOps ~]$
```

Ora passiamo alla seconda parte dove parleremo dei permessi dei file. Navighiamo in /home/analyst/lab.support.files/scripts/. Usiamo il comando ls -l per visualizzare i permessi dei file.

```

[analyst@secOps ~]$ cd lab.support.files/scripts/
[analyst@secOps scripts]$ ls -l
total 68
-rwxr-xr-x 1 analyst analyst 952 Mar 21 2018 configure_as_dhcp.sh
-rwxr-xr-x 1 analyst analyst 1153 Mar 21 2018 configure_as_static.sh
-rwxr-xr-x 1 analyst analyst 4053 Jun 18 20:09 cyberops_extended_topo_no_fw.py
-rwxr-xr-x 1 analyst analyst 5016 Jun 18 20:07 cyberops_extended_topo.py
-rwxr-xr-x 1 analyst analyst 4189 Jun 18 19:22 cyberops_topo.py
-rw-r--r-- 1 analyst analyst 2871 Mar 21 2018 cyops.mn
-rwxr-xr-x 1 analyst analyst 458 Mar 21 2018 fw_rules
-rwxr-xr-x 1 analyst analyst 70 Mar 21 2018 mal_server_start.sh
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 net_configuration_files
-rwxr-xr-x 1 analyst analyst 65 Mar 21 2018 reg_server_start.sh
-rwxr-xr-x 1 analyst analyst 189 Mar 21 2018 start_ELK.sh
-rwxr-xr-x 1 analyst analyst 86 Jun 18 20:27 start_miniedit.sh
-rwxr-xr-x 1 analyst analyst 86 Jun 19 03:16 start_pox.sh
-rwxr-xr-x 1 analyst analyst 117 Jun 19 03:31 start_snort.sh
-rwxr-xr-x 1 analyst analyst 61 Mar 21 2018 start_tftpd.sh
[analyst@secOps scripts]$

```

Il proprietario del file cyops.mn è “analyst” e anche il gruppo associato al file è “analyst”, come mostrato nelle colonne dell’output. I permessi -rw-r--r-- significano che il proprietario può leggere e scrivere il file, mentre il gruppo e tutti gli altri utenti possono solo leggerlo. Non sono concessi permessi di esecuzione a nessuno.

. Il comando touch è molto semplice e utile. Permette la creazione rapida di un file di testo vuoto. Usa il comando sottostante per creare un file vuoto nella directory /mnt:

```

[analyst@secOps scripts]$ touch /mnt/ciao.txt
touch: cannot touch '/mnt/ciao.txt': Permission denied
[analyst@secOps scripts]$

```

```

[analyst@secOps scripts]$ ls -ld /mnt/
drwxr-xr-x 2 root root 4096 Dec 9 05:44 /mnt/
[analyst@secOps scripts]$ ls -ld /mnt

```

Il file non è stato creato perché la directory /mnt non permette all’utente analyst di scrivere al suo interno. I permessi mostrati per /mnt sono drwxr-xr-x e il proprietario e il gruppo sono entrambi root. Questo significa che solo root ha permessi di lettura, scrittura ed esecuzione sulla directory, mentre tutti gli altri utenti possono solo leggere il contenuto ed entrare nella directory ma non possono creare file. Tu stai tentando di creare un file in /mnt come utente

analyst, che non ha il permesso di scrittura, quindi il comando touch viene rifiutato con "Permission denied". Il contenuto della directory /mnt è vuoto, ma questo non cambia il fatto che la directory stessa è protetta. L'opzione -d fa mostrare i permessi della directory stessa invece del contenuto, e nello specifico conferma che /mnt è di proprietà di root con permessi drwxr-xr-x, quindi l'utente analyst non può modificarla o scriverci dentro.

Il comando chmod è usato per cambiare i permessi di un file o di una directory. Come prima, monta la partizione /dev/sdb1 sulla directory /home/analyst/second_drive creata precedentemente in questo laboratorio:

```
[analyst@sec0ps scripts]$ sudo mount /dev/sdb1 ~/second_drive/  
[analyst@sec0ps scripts]$
```

Entriamo nella directory second_drive ed elenchiamo il suo contenuto:

```
[analyst@sec0ps scripts]$ cd ~/second_drive/  
[analyst@sec0ps second_drive]$ ls -l  
total 20  
drwx----- 2 root    root    16384 Mar 26  2018 lost+found  
-rw-r--r--  1 analyst analyst  183 Mar 26  2018 myFile.txt  
[analyst@sec0ps second_drive]$
```

I permessi del file myFile.txt sono rw-r--r--. Questo significa che il proprietario può leggere e scrivere il file, mentre il gruppo e tutti gli altri utenti possono solo leggerlo.

Usiamo il comando chmod per cambiare i permessi di myFile.txt.

```
[analyst@sec0ps second_drive]$ sudo chmod 665 myFile.txt  
[analyst@sec0ps second_drive]$ ls -l  
total 20  
drwx----- 2 root    root    16384 Mar 26  2018 lost+found  
-rw-rw-r-x  1 analyst analyst  183 Mar 26  2018 myFile.txt  
[analyst@sec0ps second_drive]$
```

Ora myFile.txt ha permessi rw-rw-r-x. Questo significa che il proprietario può leggere e scrivere, il gruppo può leggere e scrivere, mentre gli altri utenti possono solo leggere ed eseguire.

Il comando chmod prende i permessi nel formato ottale. In questo modo, una scomposizione di 665 è la seguente: 6 in ottale è 110 in binario. Assumendo che ogni posizione dei permessi di un file possa essere 1 o 0, 110 significa rw- (lettura=1, scrittura=1 ed esecuzione=0. Pertanto, il

comando `chmod 665 myFile.txt` cambia i permessi a: Proprietario: rw- 6 in ottale o 110 in binario) Gruppo: rw- 6 in ottale o 110 in binario) Altri: r-x 5 in ottale o 101 in binario). Il comando `chmod 777 myFile.txt` imposta i permessi a `rw-rwxrwx`, dando accesso completo a tutti gli utenti.

Il comando `chown` è usato per cambiare la proprietà di un file o di una directory. Eseguiamo il comando sottostante per rendere root il proprietario di `myFile.txt`:

```
[analyst@sec0ps second_drive]$ sudo chown analyst myFile.txt
[analyst@sec0ps second_drive]$ ls -l
total 20
drwx----- 2 root    root    16384 Mar 26  2018 lost+found
-rw-rw-r-x 1 analyst analyst  183 Mar 26  2018 myFile.txt
[analyst@sec0ps second_drive]$
```

Per cambiare sia il proprietario che il gruppo in `analyst` allo stesso tempo, usa il formato `sudo chown analyst:analyst myFile.txt`. h. Ora che `analyst` è il proprietario del file, proviamo ad accodare la parola 'test' alla fine di `myFile.txt`.

L'operazione è riuscita. Lo si vede perché dopo aver eseguito il comando: il contenuto del file, mostrato con `cat myFile.txt`, termina proprio con la parola "test". Questo è possibile perché ora il file appartiene all'utente `analyst` e i permessi del file permettono al proprietario di scrivere. Poiché `analyst` è il proprietario e ha permessi di scrittura, il comando di accodamento è stato accettato senza errori e il testo è stato aggiunto correttamente alla fine del file.

Simile ai file regolari, anche le directory portano permessi. Sia i file che le directory hanno 9 bit per i permessi del proprietario/utente, del gruppo e degli altri. Ci sono anche altri tre bit per permessi speciali: `setuid`, `setgid` e `sticky bit`, che vanno oltre lo scopo di questo laboratorio.

Torniamo alla directory `/home/analyst/lab.support.files` ed eseguiamo il comando `ls -l` per elencare tutti i file con dettagli:

```

[analyst@sec0ps second_drive]$ cd ~/lab.support.files/
[analyst@sec0ps lab.support.files]$ ls -l
total 580
-rw-r--r-- 1 analyst analyst 649 Mar 21 2018 apache_in_epoch.log
-rw-r--r-- 1 analyst analyst 126 Mar 21 2018 applicationX_in_epoch.log
drwxr-xr-x 4 analyst analyst 4096 Mar 21 2018 attack_scripts
-rw-r--r-- 1 analyst analyst 102 Mar 21 2018 confidential.txt
-rw-r--r-- 1 analyst analyst 2871 Mar 21 2018 cyops.mn
-rw-r--r-- 1 analyst analyst 75 Mar 21 2018 elk_services
-rw-r--r-- 1 analyst analyst 373 Mar 21 2018 h2_dropbear.banner
drwxr-xr-x 2 analyst analyst 4096 Apr 2 2018 instructor
-rw-r--r-- 1 analyst analyst 255 Mar 21 2018 letter_to_grandma.txt
-rw-r--r-- 1 analyst analyst 24464 Mar 21 2018 logstash-tutorial.log
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 malware
-rwxr-xr-x 1 analyst analyst 172 Mar 21 2018 mininet_services
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 openssl_lab
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 pcaps
drwxr-xr-x 6 analyst analyst 4096 Aug 15 2022 pox
-rw-r--r-- 1 analyst analyst 473363 Mar 21 2018 sample.img
-rw-r--r-- 1 analyst analyst 65 Mar 21 2018 sample.img_SHA256.sig
drwxr-xr-x 3 analyst analyst 4096 Jun 18 20:07 scripts
-rw-r--r-- 1 analyst analyst 25553 Mar 21 2018 SQL_Lab.pcap
[analyst@sec0ps lab.support.files]$

```

La differenza tra la parte iniziale della riga di malware e quella di mininet_services è che la riga di malware inizia con la lettera d, che indica una directory, mentre la riga di mininet_services inizia con un trattino, che indica un file normale. Inoltre malware ha permessi che permettono di entrare nella directory, mentre mininet_services ha permessi che includono l'esecuzione perché è un file eseguibile. I comandi chmod e chown funzionano per le directory nello stesso modo in cui funzionano per i file.

Abbiamo visto alcuni dei diversi tipi di file in Linux. Il primo carattere in ogni file elencato con il comando `ls -l` mostra il tipo di file. I tre diversi tipi di file in Linux, inclusi i loro sottotipi e caratteri, sono:

- File Regolari (-) che includono:
 - File leggibili - file di testo
 - File binari - programmi
 - File immagine
 - File compressi

- File Directory (d)
 - Cartelle
- File Speciali che includono:
 - File a Blocco (b) - File usati per accedere all'hardware fisico come punti di montaggio per accedere agli hard disk.
 - File a Carattere (c) - File che forniscono un flusso seriale di input e output. I terminali tty sono esempi di questo tipo di file.
 - File Pipe (p) - Un file usato per passare informazioni dove i primi byte in entrata sono i primi byte in uscita. Questo è anche noto come FIFO (first in first out).
 - File Link Simbolici (l) - File usati per collegarsi ad altri file o directory. Ci sono due tipi: link simbolici e hard link.
 - File Socket (s) - Questi sono usati per passare informazioni da applicazione ad applicazione per comunicare su una rete.

Usiamo il comando `ls -l` per visualizzare i file nella cartella `/home/analyst`. Nota che i primi caratteri di ogni riga sono o un `-` che indica un file o un `d` che indica una directory.

Usiamo il comando `ls -l` per visualizzare i file nella cartella `/home/analyst`. Nota che i primi caratteri di ogni riga sono o un `-` che indica un file o un `d` che indica una directory.

```
[analyst@secOps ~]$ ls -l
total 24
drwxr-xr-x 2 analyst analyst 4096 Jun 17 19:35 Desktop
drwxr-xr-x 3 analyst analyst 4096 Jun 18 20:17 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jun 18 20:17 lab.support.files
drwxr-xr-x 3 analyst analyst 4096 Jun 18 19:55 scripts
drwxr-xr-x 3 root     root     4096 Mar 26 2018 second_drive
drwxr-xr-x 5 analyst analyst 4096 Jun 18 19:27 yay
[analyst@secOps ~]$
```

Produci un elenco della directory /dev. Scorri fino al centro dell'output e nota come i file a blocco iniziano con una “b”, i file a carattere iniziano con una “c” e i file link simbolici iniziano con una “l”.

```
[analyst@secOps ~]$ ls -l /dev
total 0
crw-r--r--  1 root root      10, 235 Dec  9 04:16 autofs
drwxr-xr-x  2 root root      140 Dec  9 04:16 block
drwxr-xr-x  2 root root      100 Dec  9 04:16 bsg
crw-rw----  1 root disk     10, 234 Dec  9 04:16 btrfs-control
drwxr-xr-x  3 root root        60 Dec  9 04:16 bus
lrwxrwxrwx  1 root root        3 Dec  9 04:16 cdrom -> sr0
drwxr-xr-x  2 root root    3780 Dec  9 04:16 char
crw-----  1 root root        5,   1 Dec  9 04:16 console
lrwxrwxrwx  1 root root       11 Dec  9 04:16 core -> /proc/kcore
drwxr-xr-x  4 root root        80 Dec  9 04:16 cpu
crw-----  1 root root     10, 124 Dec  9 04:16 cpu_dma_latency
crw-----  1 root root     10, 203 Dec  9 04:16 cuse
drwxr-xr-x  7 root root      140 Dec  9 04:16 disk
drwxr-xr-x  2 root root        60 Dec  9 04:16 dma_heap
drwxr-xr-x  3 root root      100 Dec  9 04:16 dri
crw-rw----  1 root video    29,   0 Dec  9 04:16 fb0
lrwxrwxrwx  1 root root       13 Dec  9 04:16 fd -> /proc/self/fd
crw-rw-rw-  1 root root        1,   7 Dec  9 04:16 full
crw-rw-rw-  1 root root     10, 229 Dec  9 04:16 fuse
crw-----  1 root root    244,   0 Dec  9 04:16 hidraw0
crw-----  1 root root     10, 228 Dec  9 04:16 hpet
drwxr-xr-x  2 root root        0 Dec  9 04:16 hugepages
crw-----  1 root root     10, 183 Dec  9 04:16 hwrng
drwxr-xr-x  4 root root      360 Dec  9 04:16 input
crw-r--r--  1 root root        1,  11 Dec  9 04:16 kmsg
lrwxrwxrwx  1 root root       28 Dec  9 04:16 log -> /run/systemd/journal/dev-log
crw-rw----  1 root disk     10, 237 Dec  9 04:16 loop-control
drwxr-xr-x  2 root root        60 Dec  9 04:16 mapper
crw-r----- 1 root kmem        1,   1 Dec  9 04:16 mem
drwxrwxrwt  2 root root        40 Dec  9 04:16 mqueue
drwxrwxrwt  2 root root        60 Dec  9 04:16 net
```

I link simbolici in Linux sono come i collegamenti in Windows. Ci sono due tipi di link in Linux: link simbolici e hard link. La differenza tra link simbolici e hard link è che un file link simbolico punta al nome file di un altro file e un file hard link punta al contenuto di un altro file. Crea due file

```
[analyst@secOps ~]$ echo "symbolic" > file1.txt
[analyst@secOps ~]$ cat file1.txt
symbolic
[analyst@secOps ~]$ echo "hard" > file2.txt
[analyst@secOps ~]$ cat file2.txt
hard
[analyst@secOps ~]$
```

Usiamo `ln -s` per creare un link simbolico a `file1.txt`, e `ln` per creare un hard link a `file2.txt`

```
analyst@secOps ~]$ ln -s file1.txt file1symbolic
analyst@secOps ~]$ ln file2.txt file2hard
analyst@secOps ~]$
```

Usiamo il comando `ls -l` ed esamina l'elenco della directory:

```
[analyst@secOps ~]$ ls -l
total 36
drwxr-xr-x 2 analyst analyst 4096 Jun 17 19:35 Desktop
drwxr-xr-x 3 analyst analyst 4096 Jun 18 20:17 Downloads
lrwxrwxrwx 1 analyst analyst   9 Dec  9 06:23 file1symbolic -> file1.txt
-rw-r--r-- 1 analyst analyst   9 Dec  9 06:20 file1.txt
-rw-r--r-- 2 analyst analyst   5 Dec  9 06:21 file2hard
-rw-r--r-- 2 analyst analyst   5 Dec  9 06:21 file2.txt
drwxr-xr-x 9 analyst analyst 4096 Jun 18 20:17 lab.support.files
drwxr-xr-x 3 analyst analyst 4096 Jun 18 19:55 scripts
drwxr-xr-x 3 root    root    4096 Mar 26  2018 second_drive
drwxr-xr-x 5 analyst analyst 4096 Jun 18 19:27 yay
[analyst@secOps ~]$
```

Cambiamo i nomi dei file originali: `file1.txt` e `file2.txt`, e nota come influisce sui file collegati.

```
[analyst@secOps ~]$ mv file1.txt file1new.txt
[analyst@secOps ~]$ mv file2.txt file2new.txt
[analyst@secOps ~]$ cat file1symbolic
cat: file1symbolic: No such file or directory
[analyst@secOps ~]$ cat file2hard
hard
[analyst@secOps ~]$
```

Nota come file1symbolic sia ora un link simbolico rotto perché il nome del file a cui puntava, file1.txt, è cambiato, ma il file hard link file2hard funziona ancora correttamente perché punta all'inode di file2.txt e non al suo nome, che ora è file2new.txt.

Se modifichiamo il contenuto di file2new.txt, anche file2hard mostrerebbe esattamente le stesse modifiche, perché entrambi puntano allo stesso inode e quindi allo stesso contenuto fisico sul disco.

Riflessione

I permessi e la proprietà dei file sono due degli aspetti più importanti di Linux. Sono anche una causa comune di problemi. Un file con permessi o proprietà errati non sarà disponibile per i programmi che devono accedervi. In questo scenario, il programma di solito si interromperà e si incontreranno errori.